



Information Technology
Operations Specialist

PROG 1700 — Logic and Programming

Workshop 01 — Logic, Algorithms & Pseudocode

Workshop Details

Course	PROG 1700 — Logic and Programming
Instructor	Davis Boudreau
Workshop	Workshop 01 — Logic, Algorithms & Pseudocode
Week	Week 2
Value	3%
Submission	Brightspace — Workshop Reflection & Exit Evidence
Primary Tools	VS Code / Course IDE, Git, GitHub, pseudocode, IPO analysis

1. Overview

Purpose

Programming begins before code is written.

A programmer must first understand the problem, determine what information is available, identify what must be calculated or processed, and determine what the program should produce.

In this workshop, you will practice converting a problem into a logical solution before implementing that solution as programming code.

The development process for this workshop is:

Problem → Decompose → IPO → Algorithm → Pseudocode → Trace → Code → Test → Debug → Improve

The emphasis is not on writing a large amount of code.

The emphasis is on developing a solution that you can explain.

Why This Matters

One of the most common mistakes made by new programmers is beginning to write code before they understand the problem.

When this happens, students can become focused on syntax:

What command do I use?

What do I type here?

Why doesn't this line work?

Those questions matter, but they come after a more important question:

What should the program actually do?

Programming is the process of translating a logical solution into instructions that a computer can execute.

A strong programmer learns to separate two problems:

Problem 1 — Designing the solution

and

Problem 2 — Expressing the solution using programming code

If the logic is incorrect, perfectly written code can still produce the wrong result.

Learning Objectives

By the end of this workshop, you should be able to:

- interpret a programming problem statement;
- identify the required inputs, processing, and outputs;
- decompose a larger problem into smaller tasks;
- distinguish between a problem, algorithm, and program;
- design a step-by-step algorithm;
- represent an algorithm using pseudocode;
- manually trace an algorithm using sample data;
- identify logical errors before implementation;
- translate a simple algorithm into programming code;
- compare expected and actual program behaviour;
- test a program using multiple input values;
- use Git and GitHub to record completed programming work;
- reflect on how planning affects programming success.

2. Learning Outcomes Addressed

This workshop primarily addresses:

Outcome 1: Translate logic principles into programming code to solve problems.

It also continues development toward:

Outcome 2: Perform Input/Output operations within software applications to retrieve data for manipulation and/or presentation.

Outcome 7: Implement debugging techniques using an IDE.

Outcome 1 is the primary focus.

3. Before We Begin — Prior Knowledge Check

Before beginning the workshop, discuss the following questions with your group.

Do not immediately search for answers or begin coding.

Discuss

1. What is a problem in the context of programming?
2. What is an algorithm?
3. Does an algorithm need to be written using a programming language?
4. What is pseudocode?
5. Why might a programmer write pseudocode before writing code?
6. What information does a program need before it can perform a calculation?
7. What is the difference between input and output?
8. What does it mean to "process" data?
9. How could you test an algorithm without running a computer program?
10. Can a program run successfully but still be wrong?

Consider

Suppose you are asked:

Create a program that determines the total cost of purchasing several identical items.

Before writing any code:

- What information would the program need?
- What would it do with that information?
- What result should it produce?

Your instructor will use this discussion to determine the class's current understanding of programming logic.

4. Concepts & Theory

4.1 Problem Solving Before Programming

Programming is a form of structured problem solving.

A useful way to think about the process is:

Understand → Decompose → Design → Implement → Test → Improve

The programming language comes primarily into the **implementation** stage.

Before implementation, we need a logical solution.

4.2 Problem Decomposition

Large problems are easier to solve when they are divided into smaller problems.

This process is called **decomposition**.

For example:

Calculate and display the final cost of a purchase.

could be decomposed into:

1. obtain the item price;
2. obtain the quantity;
3. calculate the subtotal;
4. calculate applicable additional costs;
5. calculate the final total;
6. display the results.

Each smaller task is easier to understand than the complete problem.

As programs become larger, decomposition becomes increasingly important.

4.3 Input → Process → Output

Many programming problems can initially be analyzed using the **IPO model**.

Input

What information does the program need?

Process

What must the program do with that information?

Output

What information should the program produce?

This gives us:

Input → Process → Output

or:

I → P → O

Example

Problem:

Calculate the area of a rectangular room.

Input

- room length
- room width

Process

- multiply length by width

Output

- calculated area

Before writing code, we already understand the essential logic.

4.4 Algorithms

An **algorithm** is a finite sequence of logical steps used to solve a problem.

A good algorithm should be:

- understandable;
- ordered;
- precise enough to follow;
- finite;
- capable of producing the required result.

An algorithm is not necessarily computer code.

For example:

1. Ask the user for the room length.
2. Ask the user for the room width.
3. Multiply the length by the width.
4. Store the result as the area.
5. Display the area.

That is an algorithm.

4.5 Pseudocode

Pseudocode represents programming logic in a structured, human-readable form without requiring the exact syntax of a programming language.

For example:

```
BEGIN

    INPUT length
    INPUT width

    area = length * width

    OUTPUT area

END
```

Pseudocode helps us focus on:

What should happen?

before becoming concerned with:

How does this programming language express it?

There is no single universal pseudocode syntax.

The goal is clarity and logical structure.

4.6 Variables

Programs frequently need to remember information.

A **variable** provides a named location for a value.

For example:

```
price
quantity
subtotal
tax
total
```

Good variable names help explain the logic of an algorithm.

Compare:

```
x = a * b
```

with:

```
subtotal = price * quantity
```

The second version communicates much more about the problem being solved.

4.7 Expressions

An expression combines values, variables, and operators to produce a result.

Examples:

```
subtotal = price * quantity
```

```
average = total / numberOfValues
```

```
distance = speed * time
```

Expressions translate relationships from the problem into calculations.

4.8 Tracing an Algorithm

Before implementing an algorithm, we can manually test it.

This is called **tracing** or performing a **desk check**.

Suppose:

```
INPUT price
INPUT quantity

subtotal = price * quantity

OUTPUT subtotal
```

Using:

```
price = 12.50
quantity = 4
```

we can trace:

```
subtotal = 12.50 * 4
subtotal = 50.00
```

Expected output:

```
50.00
```

Tracing helps identify logic errors before code is written.

4.9 Expected vs. Actual Behaviour

Testing requires us to know what should happen.

Before running a program, determine:

Expected Result

Then run the program and observe:

Actual Result

Compare the two.

```
Expected Result
  ↓
  Compare
  ↑
Actual Result
```

If they differ, we have something to investigate.

This is the beginning of systematic debugging.

5. Presentation & Demonstration

Your instructor will demonstrate how to solve a small programming problem without immediately writing code.

Example Problem

A customer purchases several identical items. Create a program that asks for the price of one item and the quantity purchased, calculates the subtotal, and displays the result.

The instructor will demonstrate the following process.

Step 1 — Understand the Problem

What are we being asked to calculate?

What information is provided?

What information must the user provide?

Step 2 — Identify IPO

Input

```
price
quantity
```

Process

```
subtotal = price * quantity
```

Output

```
subtotal
```

Step 3 — Develop the Algorithm

1. Ask for item price.
 2. Store item price.
 3. Ask for quantity.
 4. Store quantity.
 5. Multiply price by quantity.
 6. Store the result.
 7. Display the result.
-

Step 4 — Represent the Algorithm as Pseudocode

```
BEGIN

    OUTPUT "Enter item price"
    INPUT price
```

```
OUTPUT "Enter quantity"
INPUT quantity

subtotal = price * quantity

OUTPUT subtotal

END
```

Step 5 — Trace the Algorithm

Use sample values.

```
price = 10
quantity = 3
```

Expected result:

```
subtotal = 30
```

Step 6 — Translate the Logic into Code

Only after the solution has been understood and tested logically should it be implemented using the course programming language.

Step 7 — Run and Test

Run the program using the same test data.

Compare the program's result with the manually calculated result.

Step 8 — Improve

Ask:

- Are the prompts understandable?
 - Are the variable names meaningful?
 - Is the output clear?
 - Does the program solve the requested problem?
-

6. Workshop Challenge — Discuss & Analyze

The Scenario

You have been asked to develop a small program for a student who wants to estimate the cost of travelling to school.

The program should ask for:

- the one-way travel distance in kilometres;
- the number of school days per week;
- the vehicle's average fuel consumption in litres per 100 kilometres;
- the current fuel price per litre.

The program should calculate and display:

- total weekly travel distance;
- estimated litres of fuel required;
- estimated weekly fuel cost.

Important

Do not begin coding yet.

Your first task is to understand the problem.

Group Analysis

Working with your group, discuss:

What do we know?

Identify the information contained in the problem statement.

What must the user provide?

Identify all required inputs.

What must the program determine?

Identify all calculations.

What should the program display?

Identify the required outputs.

What assumptions are being made?

For example:

Does "one-way travel distance" mean the student travels that distance twice per school day?

Discuss assumptions before implementing the program.

7. Activities & Tasks

Task 1 — Decompose the Problem

What

Break the travel-cost problem into smaller tasks.

Why

Decomposition makes larger problems easier to understand and implement.

How

Create a numbered list describing the major tasks your program must perform.

Do not use programming syntax.

Think about the solution as a sequence of actions.

Checkpoint 1

Compare your decomposition with another group.

Do both solutions contain the same essential tasks?

Task 2 — Create an IPO Analysis

What

Identify the inputs, processing, and outputs.

Why

IPO analysis helps establish the flow of information through the program.

How

Complete the following:

Input	Process	Output

Your processing column should identify the required calculations, not programming commands.

Checkpoint 2

Before proceeding, be prepared to explain how every output is derived from the supplied inputs.

Task 3 — Design the Algorithm

What

Create a step-by-step algorithm for the program.

Why

The algorithm provides a logical plan before implementation begins.

How

Write a numbered sequence of instructions.

Your algorithm should begin with obtaining the required information and finish with displaying the required results.

Ask:

Could another student follow these instructions without asking me what I meant?

If not, improve the algorithm.

Task 4 — Convert the Algorithm to Pseudocode

What

Represent your algorithm using structured pseudocode.

Why

Pseudocode provides a bridge between human reasoning and programming code.

How

Use meaningful variable names.

Your pseudocode should clearly identify:

- input;
- calculations;
- stored results;
- output.

Do not simply copy programming-language syntax.

Checkpoint 3

Exchange pseudocode with another student.

Without explanation from the author, determine:

- what information enters the algorithm;
- what calculations occur;
- what results are produced.

If the logic cannot be understood, revise it.

Task 5 — Trace the Algorithm

What

Manually execute your pseudocode using sample data.

Why

Tracing allows you to test your logic before implementation.

How

Use the following test data:

```
One-way distance = 20 km
School days = 5
Fuel consumption = 8 L/100 km
Fuel price = $1.50/L
```

Before writing code, calculate the expected results manually.

Create a trace table similar to:

Variable	Value
oneWayDistance	
schoolDays	
weeklyDistance	
fuelConsumption	
litresRequired	
fuelPrice	
weeklyFuelCost	

Checkpoint 4

Compare your expected results with another student.

If your answers differ, investigate the logic before proceeding.

Task 6 — Implement the Program

What

Translate your completed algorithm into programming code.

Why

This demonstrates the connection between logical design and program implementation.

How

Create the program using the course programming environment.

Your program should:

1. display meaningful prompts;
2. retrieve the required input;
3. store values using meaningful variable names;
4. perform the required calculations;
5. store calculated results;
6. display clearly labelled output.

Important

Keep your pseudocode available while coding.

When you write each part of the program, identify which part of the algorithm you are implementing.

Task 7 — Test the Program

What

Determine whether your implementation produces the expected results.

Why

A program running without an error does not prove that it is correct.

How

First use the same values from your manual trace.

Record:

Test	Expected	Actual	Pass/Fail
Weekly distance			
Fuel required			
Weekly fuel cost			

If the expected and actual values match, create at least **two additional test cases**.

For each test:

1. determine the expected result manually;
2. run the program;
3. record the actual result;
4. compare the results.

Checkpoint 5

Show evidence of at least three completed tests.

Task 8 — Debug and Improve

What

Investigate any difference between expected and actual behaviour.

Why

Debugging should be based on evidence rather than random code changes.

How

If your program produces an unexpected result:

1. reproduce the problem;
2. identify the calculation involved;
3. inspect the corresponding pseudocode;
4. inspect the corresponding code;
5. use program output or the IDE debugger to inspect values;
6. identify where the actual value first differs from the expected value;
7. correct the problem;
8. rerun the test.

Even if your program works correctly, use the debugger to step through at least part of the program and observe values changing during execution.

Checkpoint 6

Identify one variable and explain:

Where does its value come from?

Where is it used?

How did you verify that the value was correct?

Task 9 — Review Code Quality

What

Review your completed program before submission.

Why

Working code should also be understandable code.

How

Review:

- variable names;
- indentation;
- prompts;
- output labels;
- unnecessary code;
- calculation clarity.

Ask another student to look at your program.

Can they understand its purpose without you explaining every line?

Task 10 — Commit and Push

What

Record your completed work using Git and GitHub.

Why

Version control is part of the normal development workflow in PROG 1700.

How

Before committing:

1. save all files;
2. run the completed program;
3. verify your tests;

4. inspect Git status;
5. review your changes.

Create a meaningful commit describing the completed work.

Push the commit to GitHub.

Verify that the new commit and source code appear in the remote repository.

Checkpoint 7

Show the completed repository and latest commit.

8. Checkpoints & Verification

Before Workshop 01 is considered complete, verify:

- ☐ Problem has been decomposed into smaller tasks.
- ☐ IPO analysis is complete.
- ☐ Algorithm has been written.
- ☐ Pseudocode represents the complete solution.
- ☐ Meaningful variable names are used.
- ☐ Algorithm has been manually traced.
- ☐ Expected results were determined before coding.
- ☐ Pseudocode has been translated into working code.
- ☐ Program accepts the required input.
- ☐ Program performs the required calculations.
- ☐ Program produces clearly labelled output.
- ☐ At least three test cases have been completed.
- ☐ Expected and actual results have been compared.
- ☐ IDE debugging tools have been used.
- ☐ Code has been reviewed for readability.
- ☐ Changes have been committed using Git.
- ☐ Commit has been pushed and verified on GitHub.

9. Reflection Questions

Complete the following individually.

1. Problem Decomposition

How did breaking the problem into smaller tasks affect your ability to understand the programming challenge?

2. IPO

Identify one example of input, processing, and output from your program.

Explain how they are connected.

3. Algorithm vs. Code

What is the difference between an algorithm and programming code?

4. Pseudocode

Did your pseudocode change before or during implementation?

If so, what changed and why?

5. Testing

Why did you determine expected results before running the program?

6. Debugging

Describe one value you inspected while running or debugging your program.

How did you determine whether the value was correct?

7. Problem Solving

Which stage was most difficult for you?

Problem → Decompose → IPO → Algorithm → Pseudocode → Trace → Code → Test → Debug

Explain why.

8. Improvement

If you were given a similar programming problem tomorrow, what would you do differently?

10. Submission Guidelines

Submit your **Workshop 01 — Logic, Algorithms & Pseudocode Reflection & Exit Evidence** through Brightspace as directed by your instructor.

Your programming work must also be committed and pushed to the appropriate GitHub repository.

Your submission should include:

- completed IPO analysis;
- algorithm;
- pseudocode;
- trace evidence;
- evidence of testing;
- reflection responses;

- any additional exit evidence requested by your instructor.

Your GitHub repository should contain the completed program and meaningful development history.

Do not submit passwords, authentication tokens, private keys, or other credentials.

11. Evaluation Criteria

Workshop 01 is evaluated using the standard workshop criteria.

Criteria	Evidence
A — Attendance	Present and actively engaged during the workshop session.
P — Participation	Participates in problem analysis, group discussion, peer review, testing, and workshop activities.
T — Task Progress	Completes the IPO analysis, algorithm, pseudocode, trace, implementation, testing, debugging, and Git/GitHub workflow.
F — Focus & Professional Conduct	Uses workshop time effectively, follows the development process, works professionally, and appropriately supports classmates.
R — Reflection / Exit Evidence	Provides thoughtful reflection and evidence demonstrating understanding of the relationship between logic, algorithms, code, testing, and debugging.

Workshop Value: 3%

The emphasis is not simply on whether the final program runs.

You should be able to demonstrate and explain the **problem-solving process used to produce it**.

12. Completion & Exit Evidence

Before leaving Workshop 01, you should be able to demonstrate the complete problem-solving workflow:

Problem

↓

Decompose

↓

Identify Input → Process → Output

↓

Design Algorithm

↓

Write Pseudocode

↓

Trace with Sample Data

↓

Determine Expected Results

↓

Implement as Code

↓

Run

↓

Compare Expected vs. Actual

↓

Debug

↓

Improve

↓

Commit

↓

Push**Exit Challenge**

Your instructor may provide a short new problem.

Before writing code, identify:

1. the required input;
2. the required processing;
3. the expected output;
4. the major steps in the algorithm.

Then explain how you would test the solution.

The objective is not to immediately produce code.

The objective is to demonstrate that you know **how to begin solving a programming problem**.

Workshop 01 Complete

The central lesson from this workshop should carry forward throughout PROG 1700:

Do not begin with code. Begin with the problem.

Our development process is:

Problem → Logic → Algorithm → Code → Test → Debug → Improve

As programming problems become more complex, the syntax will change and new programming structures will be introduced.

The problem-solving process remains.